

Chapitre 2 - C#, SQL Server, Sans Design Pattern

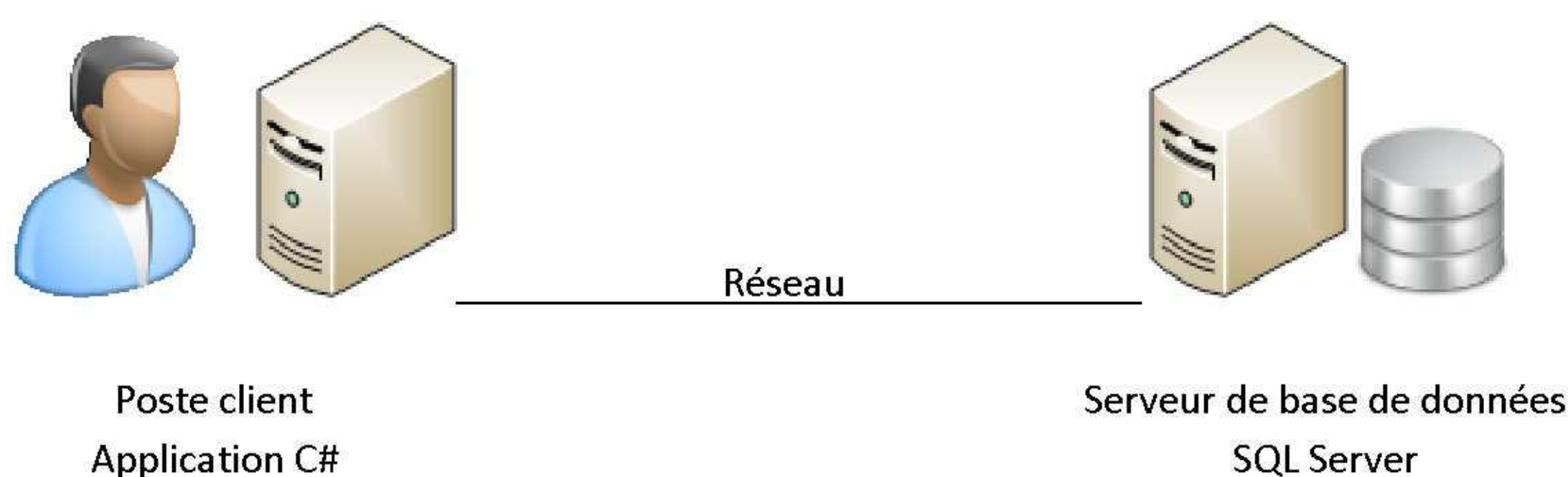
I.	Introduction.....	1
1.	Contexte	1
2.	Objectifs	1
3.	Cahier des charges à respecter.....	2
II.	Etude de la phase 1.....	2
1.	Les données	2
2.	Les traitements - Maquette de l'application	2
III.	Réalisation de la phase 1.....	3
1.	Réalisation de l'interface :.....	3
2.	Code de l'application – Onglet Salariés.....	4
3.	Code de l'application – Onglet Absences.....	7
IV.	Conclusion	8
V.	PPE	9

I. Introduction

1. Contexte

Dans l'entreprise EPOKA, la DRH gère les absences du personnel. On se propose de réaliser une application de suivi de ces absences, en stockant les données dans une base SQL Server.

L'architecture 2 tiers retenue est la suivante :



2. Objectifs

Cette application sera présentée dans 4 versions différentes, pour mettre en évidence différentes techniques de construction d'une application autour d'une base de données en client lourd (C#, Delphi, VB, Java, ...), avec leurs avantages respectifs.

3. Cahier des charges à respecter

A partir d'une fenêtre proposant des onglets, on veut pouvoir gérer entièrement les données de cette application (consultation, ajout, modification et suppression). L'ergonomie doit être satisfaisante (bouton désactivé si l'action ne peut être lancée notamment).

Les absences seront accessibles uniquement par salarié, et seront présentées par catégories (maladies, congés, ...).

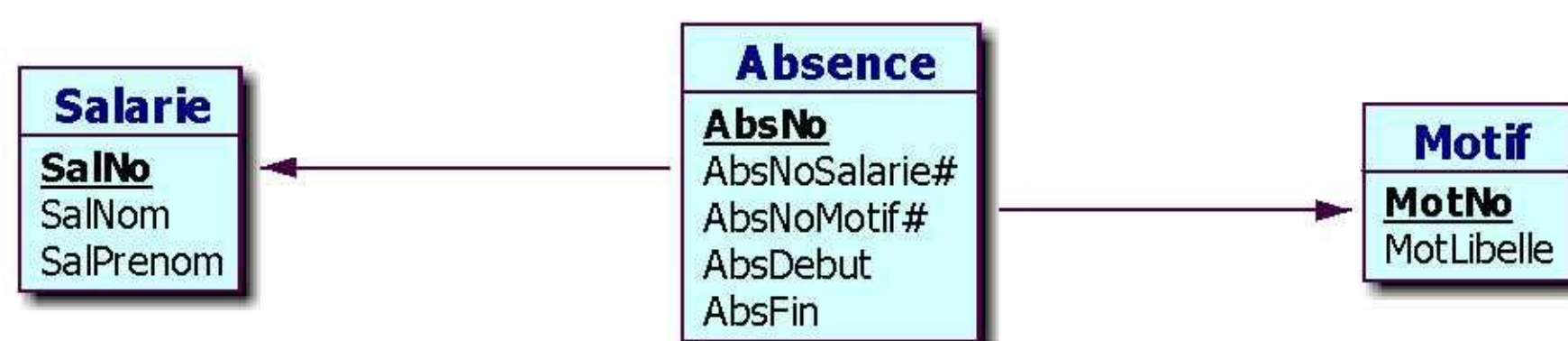
On envisage de réaliser le développement en 2 phases :

- Phase 1 :
 - L'onglet *Salariés* sera complètement fonctionnel ;
 - L'onglet *Absences* se limitera à de la consultation ;
 - L'onglet *Motifs* ne sera pas développé ;
- Phase 2 :
 - Les onglets *Absences* et *Motifs* seront entièrement fonctionnels ;

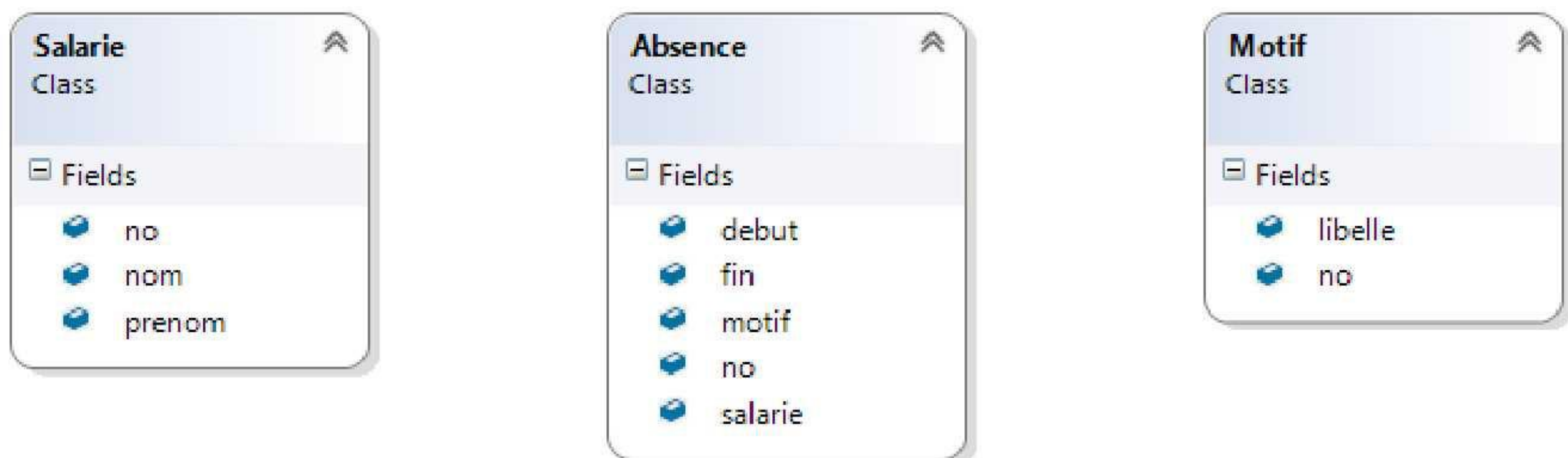
II. Etude de la phase 1

1. Les données

Voici le schéma de la base de données retenue :



Dans *Visual Studio*, un diagramme de classes métier correspondant a été créé :



Ce diagramme a créé les classes :

Fichier *Salarie.cs* :

```
public class Salarie
{
    public int no;
    public String nom;
    public String prenom;
}
```

Fichier *Absence.cs* :

```
public class Absence
{
    public int no;
    public DateTime debut;
    public DateTime fin;
    public Motif motif;
    public Salarie salaire;
}
```

Fichier *Motif.cs* :

```
public class Motif
{
    public int no;
    public String libelle;
}
```

2. Les traitements - Maquette de l'application

L'onglet *Salariés* :

Absences des salariés

Salariés | Absences | Motifs

No	Nom	Prénom
1	BERNARD	Evans
2	FUERTES	Vivien
3	GAUDIN	Lisa
4	POUJADE	Colette

Nom: FUERTES Prénom: Vivien

L'onglet *Absences* :

Absences des salariés

Salariés | Absences | Motifs

Salarié: FUERTES Vivien

Du	Au	Durée
Maladie		
mer. 01/06/2016	jeu. 02/06/2016	2 jour(s)
Congés annuels		
ven. 15/07/2016	sam. 30/07/2016	16 jour(s)

III. Réalisation de la phase 1

Dans l'application *Windows Forms* :

- Un diagramme de classe a été créé ;
- Les classes *Salarie*, *Absence*, *Motif* ont été générées ;

1. Réalisation de l'interface :

Dans l'unique fenêtre de l'application :

- Nommer la fenêtre *FMere* et son fichier principal *FMere.cs* ;
- La fenêtre doit être redimensionnable (*FormBorderStyle*) ;
- Placer initialement la fenêtre au centre de l'écran (*StartPosition*) ;
- Lui donner le titre *Absences des salariés* (*Text*) ;

- Placer un objet boîte à onglets (*TabControl*) ;
- Le nommer *tcOnglets* ;
- Ancrer l'objet à ses 4 bords pour suivre le redimensionnement de la fenêtre (*Anchor*) ;
- Créer les 3 onglets par le menu accessible en haut à droite quand le *TabControl* est sélectionné ;
- Nommer les onglets (objets *TabPage*) : *tpSalaries*, *tpAbsences*, *tpMotifs* ;
- Affecter leurs libellés (*Text*) : *Salariés*, *Absences*, *Motifs* ;

Dans l'onglet *Salariés* :

- Ajouter une *ListView*, nommée *lvSalaries* ;
- L'ancrer sur les 4 bords ;
- Définir son apparence (*View*) à *Détails* ;
- Permettre la sélection par la ligne entière (*FullRowSelect*) ;
- Définir les colonnes (menu spécifique en haut à droite de l'objet) :
 - 3 colonnes (noms quelconques) ;
 - Libellés (*Text*) : *No*, *Nom*, *Prénom* ;
 - Largeurs (*Width*) : 60, 150, 150 ;
- Déposer 2 *Labels* et 2 *TextBox* ;
- Nommer les *TextBox* : *tbNom* et *tbPrenom* ;
- Laisser les labels avec leur nom générique ;
- Définir leur libellé (*Text*) avec *&Nom* et *&Prénom* ;
- Créer 4 boutons (*Button*) nommés *bAjouterSalarie*, *bModifierSalarie*, *bSupprimerSalarie*, *bAbsencesSalarie* ;
- Définir leurs libellés (*Text*) avec *&Ajouter*, *&Modifier*, *&Supprimer*, *&Absences* ;
- Ancrer sur la gauche et le bas les objets de la partie inférieure : 2 *Labels*, 2 *TextBox*, 4 *Buttons* ;
- Intervenir sur l'ordre de tabulation (*TabIndex*) ;

Dans l'onglet *Absences* :

- Déposer un *Label*, avec le libellé *&Salarié* ;
- Ajouter une *ComboBox*, nommée *cbSalarie* ;
- Définir son style (*DropDownStyle*) à *DropDownList* pour empêcher que la liste soit saisissable ;
- Ajouter un *ListView* nommé *lvAbsences* ;
- L'ancrer sur les 4 bords ;
- Définir son apparence (*View*) à *Détails* ;
- Permettre la sélection par la ligne entière (*FullRowSelect*) ;
- Créer 3 colonnes : *Du*, *Au*, *Durée*. Largeurs 100, 100, 90 ;

Fenêtre de l'application :

- Imposer une taille minimale de la fenêtre (*MinimumSize*) ;

2. Code de l'application – Onglet Salariés

Normes de codage en C# :

C# utilise la notation *PascalCase* pour les noms de méthodes et de classes, et *camelCase* pour les noms de propriétés (préférence non bloquante) alors que Java utilise *camelCase* pour les propriétés et méthodes (préférence bloquante).

Cette onglet gère la liste des salariés, mais aussi l'ajout, la modification et la suppression, avec à chaque fois rafraîchissement de la liste. On gère aussi le fait d'activer les boutons seulement quand cela a un sens, par exemple pas de suppression sans sélection dans la liste.

- Sur le chargement de la fenêtre (*Load*), il faut charger la liste des salariés dans l'onglet *Salariés*, mais aussi dans l'onglet *Absences* :

```
private void FMere_Load(object sender, EventArgs e)
{
    majListeSalaries();
}
```

- Ce chargement est aussi appelé après tout ajout, modification, suppression. Dans *MajListeSalaries()*, il faut :
 - Effacer les deux listes si elles sont déjà remplies ;
 - Se connecter à SQL Server ;
 - Lire tous les salariés ;
 - Créer pour chacun une ligne dans chaque liste et l'ajouter à la liste ;
 - Libérer le curseur et la connexion ;

```
private void majListeSalaries()
{
    // Effacer la liste de l'onglet Salaries
    lvSalaries.Items.Clear();
    // Effectuer la dé-sélection, non produite automatiquement
    lvSalaries_SelectedIndexChanged(null, null);

    // Effacer aussi la liste déroulante des salariés de l'onglet Absences
    cbSalarie.Items.Clear();
    // Effectuer la dé-sélection, non produite automatiquement
    cbSalarie_SelectedIndexChanged(null, null);

    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV13; "
        + "Initial Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    SqlCommand sqlCmd = new SqlCommand("SELECT * FROM Salarie", connexion);
    SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
    while (sqlDataReader.Read())
    {
        // Remplissage de la liste de l'onglet Salaries
        ListViewItem lvi = new ListViewItem();
        int salNo = (int)sqlDataReader["SalNo"];
        lvi.Text = salNo.ToString();
        lvi.SubItems.Add((String)sqlDataReader["SalNom"]);
        lvi.SubItems.Add((String)sqlDataReader["SalPrenom"]);
        lvSalaries.Items.Add(lvi);

        // Remplissage de la liste déroulante de l'onglet Absences
        ComboBoxItem cbi = new ComboBoxItem();
        cbi.code = salNo.ToString();
        cbi.libelle = (String)sqlDataReader["SalNom"] + " " +
            (String)sqlDataReader["SalPrenom"];
        cbSalarie.Items.Add(cbi);
    }
    sqlDataReader.Close();
    connexion.Close();
}
```

- La classe *ComboBoxItem* est une classe associant un code et un libellé, mais n'affichant que le libellé (idéal pour les listes déroulantes :

```
public class ComboBoxItem
{
    public string code;
    public string libelle;
```



```

public override string ToString()
{
    return libelle;
}
}

```

- La sélection d'un salarié entraîne la recopie des valeurs dans les champs et la mise à jour des boutons :

```

private void lvSalaries_SelectedIndexChanged(object sender, EventArgs e)
{
    if (lvSalaries.SelectedItems.Count > 0)
    {
        tbNom.Text = lvSalaries.SelectedItems[0].SubItems[1].Text;
        tbPrenom.Text = lvSalaries.SelectedItems[0].SubItems[2].Text;
    }
    else
    {
        tbNom.Text = "";
        tbPrenom.Text = "";
    }
    majBoutonsSalaries();
}

```

- La mise à jour de l'état des boutons est liée à la sélection ou non d'une ligne, et au remplissage ou pas des champs :

```

private void majBoutonsSalaries()
{
    bAjouterSalarie.Enabled = (tbNom.Text != "" && tbPrenom.Text != "");
    bSupprimerSalarie.Enabled = lvSalaries.SelectedItems.Count > 0;
    bModifierSalarie.Enabled = bAjouterSalarie.Enabled && bSupprimerSalarie.Enabled;
    bAbsencesSalarie.Enabled = bSupprimerSalarie.Enabled;
}

```

- La mise à jour des boutons est aussi déclenchée suite à modification des champs de saisie :

```

private void tbNom_TextChanged(object sender, EventArgs e)
{
    majBoutonsSalaries();
}

private void tbPrenom_TextChanged(object sender, EventArgs e)
{
    majBoutonsSalaries();
}

```

- Les clics sur les boutons *bAjouterSalarie*, *bModifierSalarie*, *bSupprimerSalarie* déclenchent une requête SQL sans curseur :

```

private void bAjouterSalarie_Click(object sender, EventArgs e)
{
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV13; Initial Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    SqlCommand sqlCmd = new SqlCommand("INSERT INTO Salarie (SalNom,SalPrenom) VALUES ('" + tbNom.Text + "', '" + tbPrenom.Text + "')", connexion);
    sqlCmd.ExecuteNonQuery();
    connexion.Close();
    majListeSalaries();
}

```



```
private void bModifierSalarie_Click(object sender, EventArgs e)
{
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV13; Initial
Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    SqlCommand sqlCmd = new SqlCommand("UPDATE Salarie SET SalNom='" + tbNom.Text + "',
SalPrenom='" + tbPrenom.Text + "' WHERE SalNo=" + lvSalaries.SelectedItems[0].Text, connexion);
    sqlCmd.ExecuteNonQuery();
    connexion.Close();
    majListeSalaries();
}
```

```
private void bSupprimerSalarie_Click(object sender, EventArgs e)
{
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV13; Initial
Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    SqlCommand sqlCmd = new SqlCommand("DELETE FROM Salarie WHERE SalNo=" +
lvSalaries.SelectedItems[0].Text, connexion);
    sqlCmd.ExecuteNonQuery();
    connexion.Close();
    majListeSalaries();
}
```

Le clic sur le bouton *bAbsencesSalarie* sélectionne le salarié dans l'onglet *Absences*, ce qui provoquera l'affichage de ses absences. L'onglet *tpAbsences* est ensuite sélectionné :

```
private void bAbsencesSalarie_Click(object sender, EventArgs e)
{
    cbSalarie.Text = lvSalaries.SelectedItems[0].SubItems[1].Text + " "
+ lvSalaries.SelectedItems[0].SubItems[2].Text;
    tcOnglets.SelectedTab = tpAbsences;
}
```

3. Code de l'application – Onglet Absences

- La sélection dans la liste déroulante provoque le rechargement des absences du salarié :

```
private void cbSalarie_SelectedIndexChanged(object sender, EventArgs e)
{
    majListeAbsences();
}
```

- Cette mise à jour de la liste déroulante nécessite de :
 - Effacer la liste si elle était déjà remplie ;
 - Se connecter à SQL Server ;
 - Lire tous les motifs ;
 - Créer pour chacun un groupe ;
 - Refermer le curseur ;
 - Lire toutes les absences avec leur motif pour le salarié sélectionné ;
 - Pour chacune préparer une ligne de la liste avec début, fin et durée ;
 - Rattacher la ligne au groupe (motif) correspondant ;
 - Ajouter la ligne à la liste ;
 - Libérer le curseur et la connexion ;


```

private void majListeAbsences()
{
    lvAbsences.Items.Clear(); // Effacer la liste de l'onglet Absences

    // Remplir les ListViewGroup avec les motifs
    SqlConnection connexion = new SqlConnection();
    connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV13; "
        + "Initial Catalog=Epoka_Absences; Integrated Security=SSPI;";
    connexion.Open();
    SqlCommand sqlCmd = new SqlCommand("SELECT * FROM Motif", connexion);
    SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
    while (sqlDataReader.Read())
    {
        String motLibelle = (String)sqlDataReader["MotLibelle"];
        ListViewGroup lvg = new ListViewGroup(motLibelle);
        lvAbsences.Groups.Add(lvg);
    }
    sqlDataReader.Close();

    if (cbSalarie.SelectedItem != null)
    {
        // Remplir les ListViewItem avec les absences du salarié,
        // en les rattachant au bon ListViewGroup (motif)
        sqlCmd.CommandText = "SELECT * FROM Absence, Motif WHERE AbsNoMotif=MotNo "
            + "AND AbsNoSalarie=" + (cbSalarie.SelectedItem as ComboBoxItem).code.ToString();
        sqlDataReader = sqlCmd.ExecuteReader();
        while (sqlDataReader.Read())
        {
            ListViewItem lvi = new ListViewItem();
            int absNo = (int)sqlDataReader["AbsNo"];
            lvi.Text = absNo.ToString();
            DateTime debut = (DateTime)sqlDataReader["AbsDebut"];
            lvi.SubItems.Add(debut.ToString("ddd dd/MM/yyyy"));
            DateTime fin = (DateTime)sqlDataReader["AbsFin"];
            lvi.SubItems.Add(fin.ToString("ddd dd/MM/yyyy"));
            TimeSpan ts = fin - debut;
            lvi.SubItems.Add((ts.Days + 1).ToString() + " jour(s)");
            // Rechercher le groupe :
            foreach (ListViewGroup lvg in lvAbsences.Groups)
            {
                if (lvg.Header.ToString() == (String)sqlDataReader["MotLibelle"])
                {
                    lvi.Group = lvg;
                }
            };
            lvAbsences.Items.Add(lvi);
        }
        sqlDataReader.Close();
        connexion.Close();
    }
}

```

IV. Conclusion

Un code facile à comprendre (tout est là, visible), mais :

- Le code est redondant (connexion et accès aux données éparpillés), difficile à maintenir ;
- Absence de sécurité dans les requêtes SQL (toute requête peut être passée, l'injection SQL est possible, ...) ;
- Absence de sécurité dans le code (aucune erreur n'est interceptée) ;
- La migration vers un autre support de stockage (fichiers XML, autre SGBD, ...) n'est pas facilitée ;

Solutions / Réponses :

- au code redondant : Organisation en classe DAO (chapitre 4), Utilisation d'un ORM (chapitre 6) ;

- à l'absence de sécurité dans les requêtes SQL : Utilisation de procédures stockées (chapitre 5) ;
- à l'absence de sécurité dans le code : Utilisation de blocs *try ... catch ... finally* (chapitre 3) ;
- à la migration difficile vers un autre support de stockage : Classes DAO et ORM ;

V. PPE

Vous êtes chargé de réaliser la phase 2.

Conseil : Commencer par réaliser l'onglet *motifs*, puis compléter l'onglet *Absences*.